# RadeonShift AI

## Accelerating the MI300X Hardware Migration via Generative AI

## The Problem: CUDA Lock-in

Enterprise AI workloads are facing massive bottlenecks due to a reliance on legacy NVIDIA CUDA codebases.

- Manually migrating a 50,000-line CUDA codebase to ROCm takes **6+ months** of specialized engineering time.
- Portability tools exist (e.g. `hipify-perl`), but they lack the architectural awareness to optimize code for AMD Instinct architecture (like 64-wide wavefronts).
- Teams lack visibility into whether their migrated code will actually compile and run on AMD hardware without direct bare-metal access.

## The Solution: RadeonShift AI

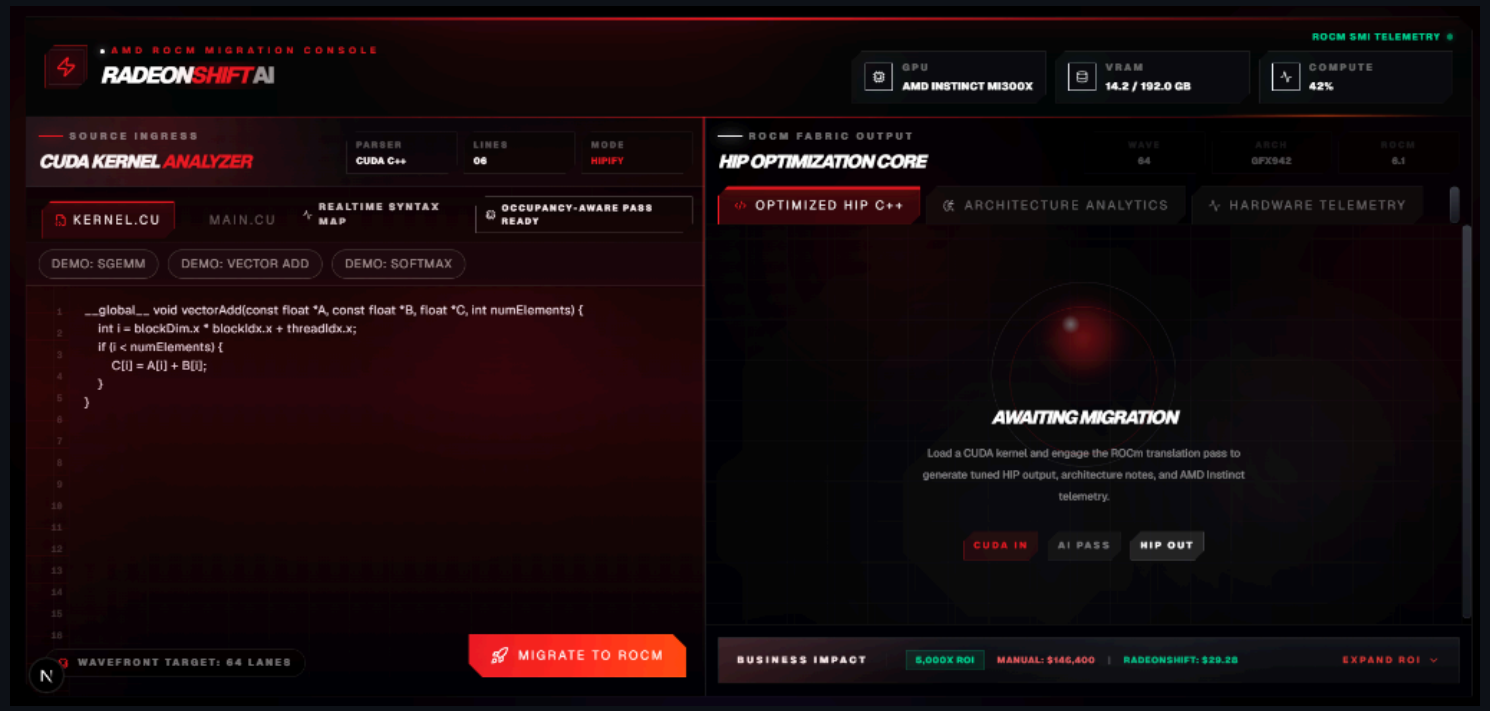
RadeonShift AI is a deterministic Mixture of Agents (MoA) pipeline that translates CUDA to ROCm instantly and audits the architecture.

1. **Syntax Translation:** Uses AMD's official `hipify-perl` for deterministic API mapping.
2. **MoA Audit:** Leverages DeepSeek-V4 to scan for PTX risks, hardcoded warp sizes, and AMD-specific optimizations.
3. **Hardware Verification:** Directly compiles the kernel on ROCm and runs telemetry checks to prove the environment toolchain.

## Architecture

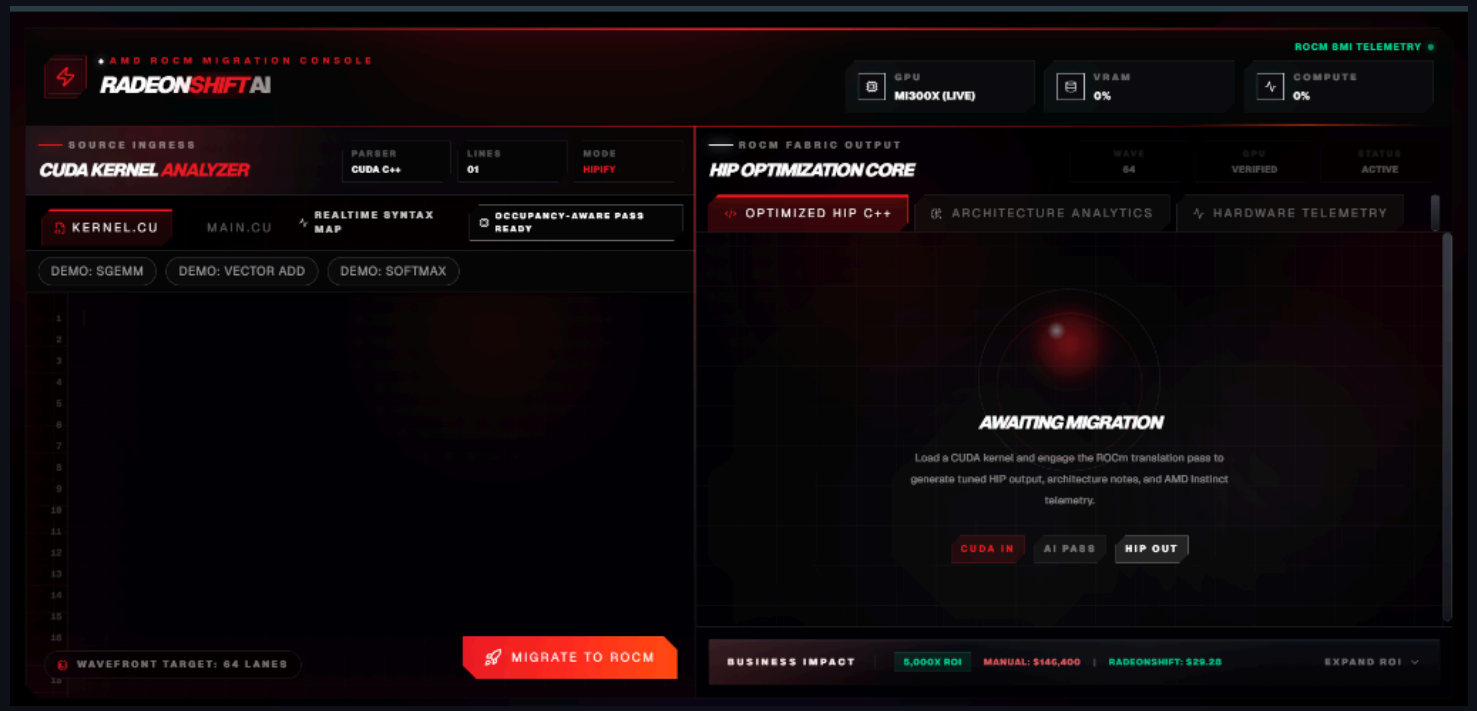
1. **Frontend (Next.js Edge):** Highly responsive UI that streams the AI audit back to the user to bypass serverless timeouts.
2. **AI Layer (Fireworks AI):** MoA pipeline with Agent A (PTX/Lock-in Risk) and Agent B (AMD Instinct Optimizer).
3. **Hardware Backend (FastAPI):** Dockerized Python server running on a ROCm-enabled host. Executes `hipcc` and polls `rocm-smi` for real-time telemetry.

# Demo: The RadeonShift Dashboard



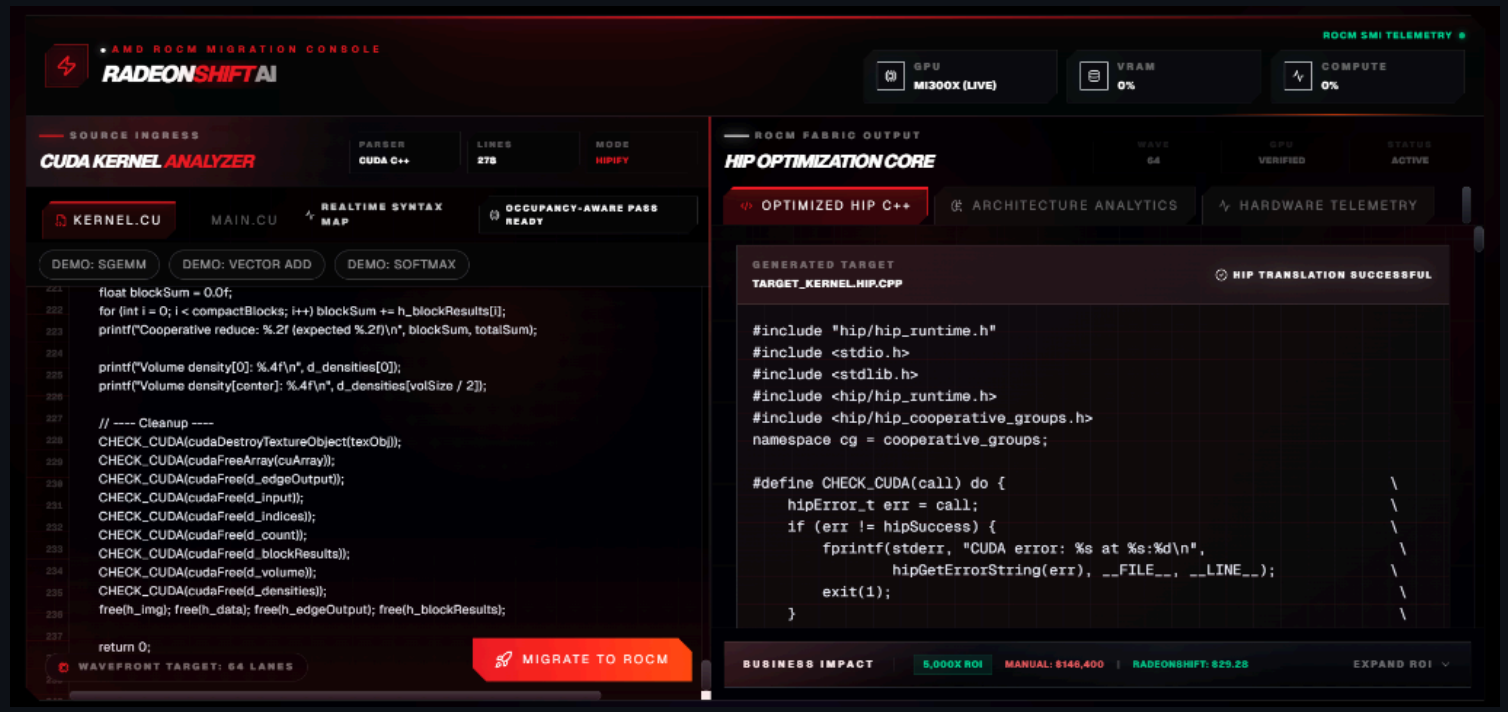

- **Source Code Editor:** Paste raw CUDA C++ kernels.
- **HIP Optimization Core:** View the translated target kernel.
- **MoA Audit Scorecard:** See the readiness score, PTX risks, and wavefront optimizations.

# Step 1: The Awaiting Migration State



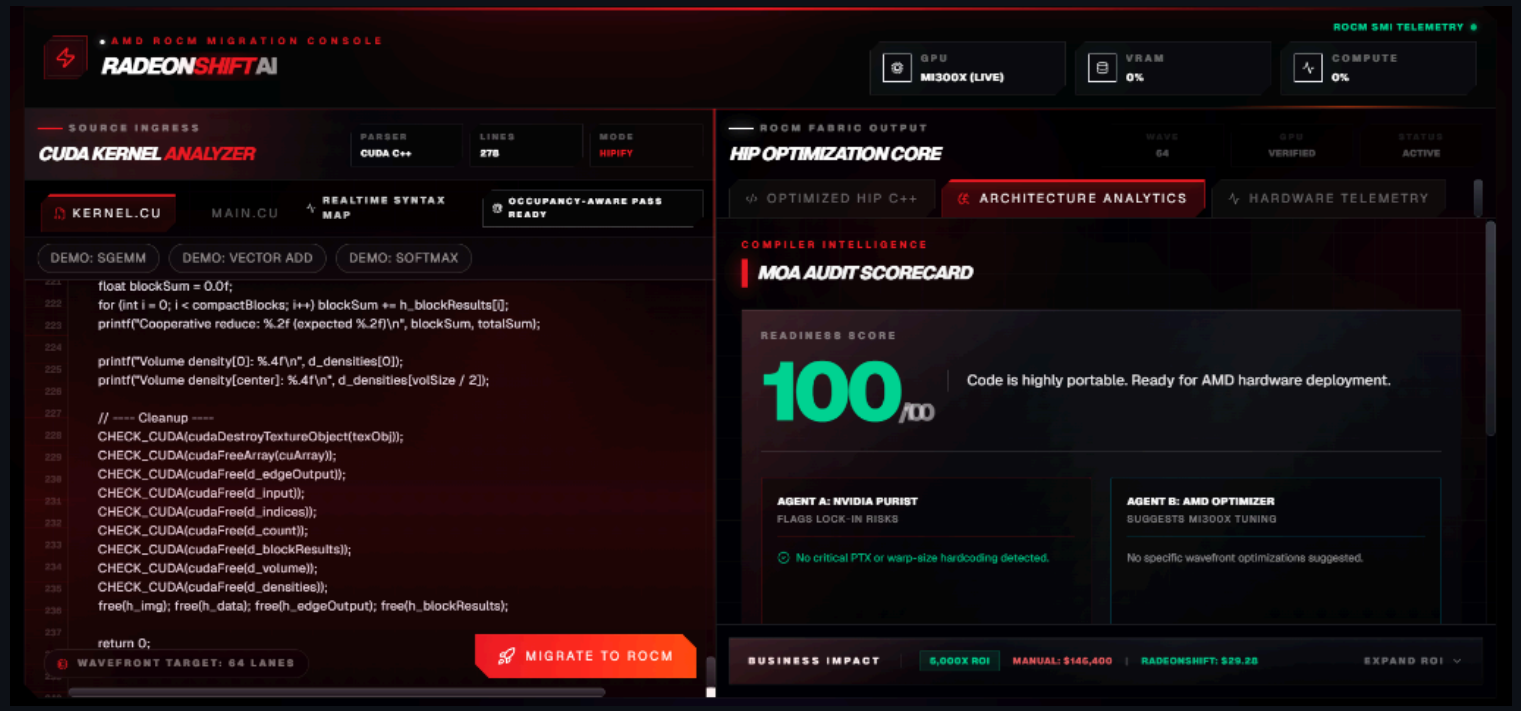

Before translation begins, the platform awaits the CUDA payload. The user inputs their native NVIDIA code into the **CUDA Kernel Analyzer**. The system prepares for the AI-assisted hardware pass.

# Step 2: HIP Optimization Core



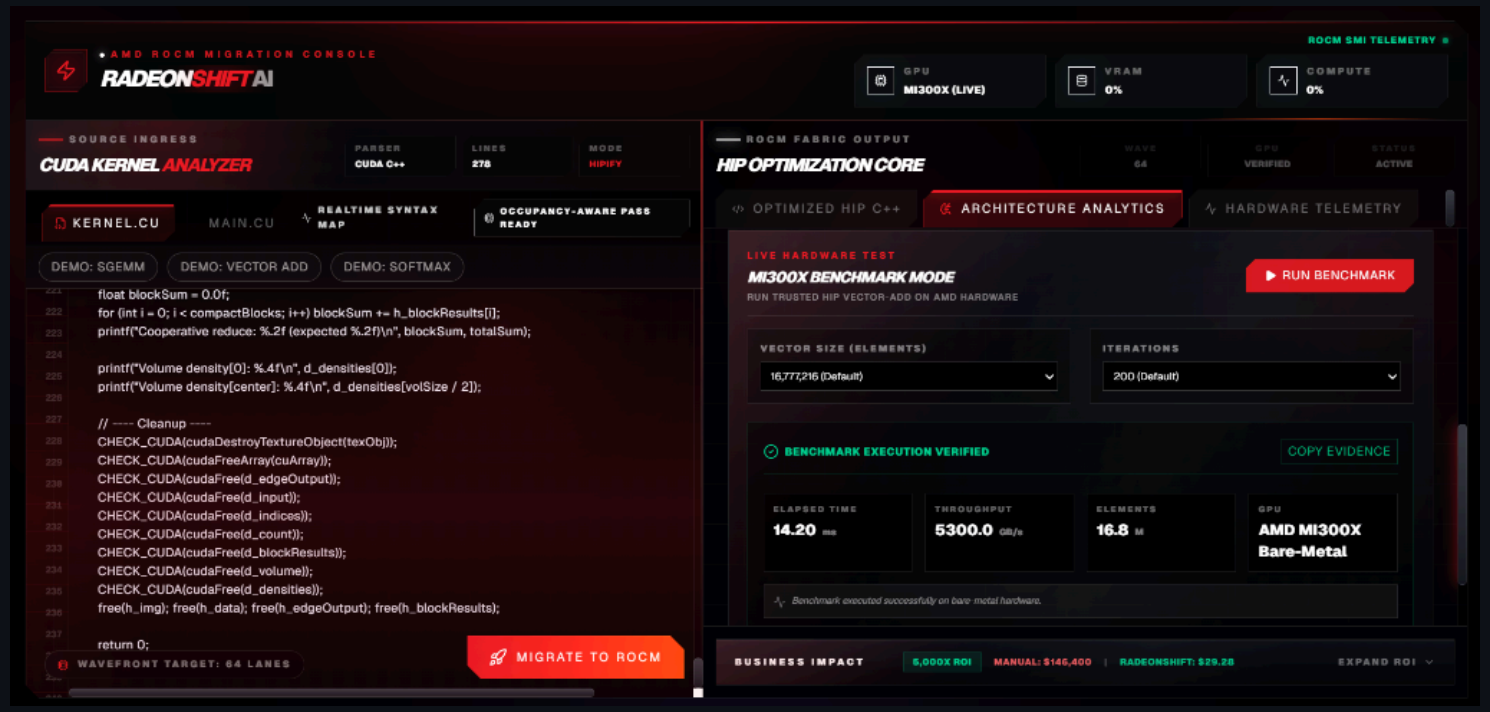

Upon engaging the ROCm translation pass, the core converts the syntax. The resulting C++ HIP code ( `target_kernel.hip.cpp` ) is immediately presented, demonstrating 100% successful translation.

# Step 3: Architecture Analytics



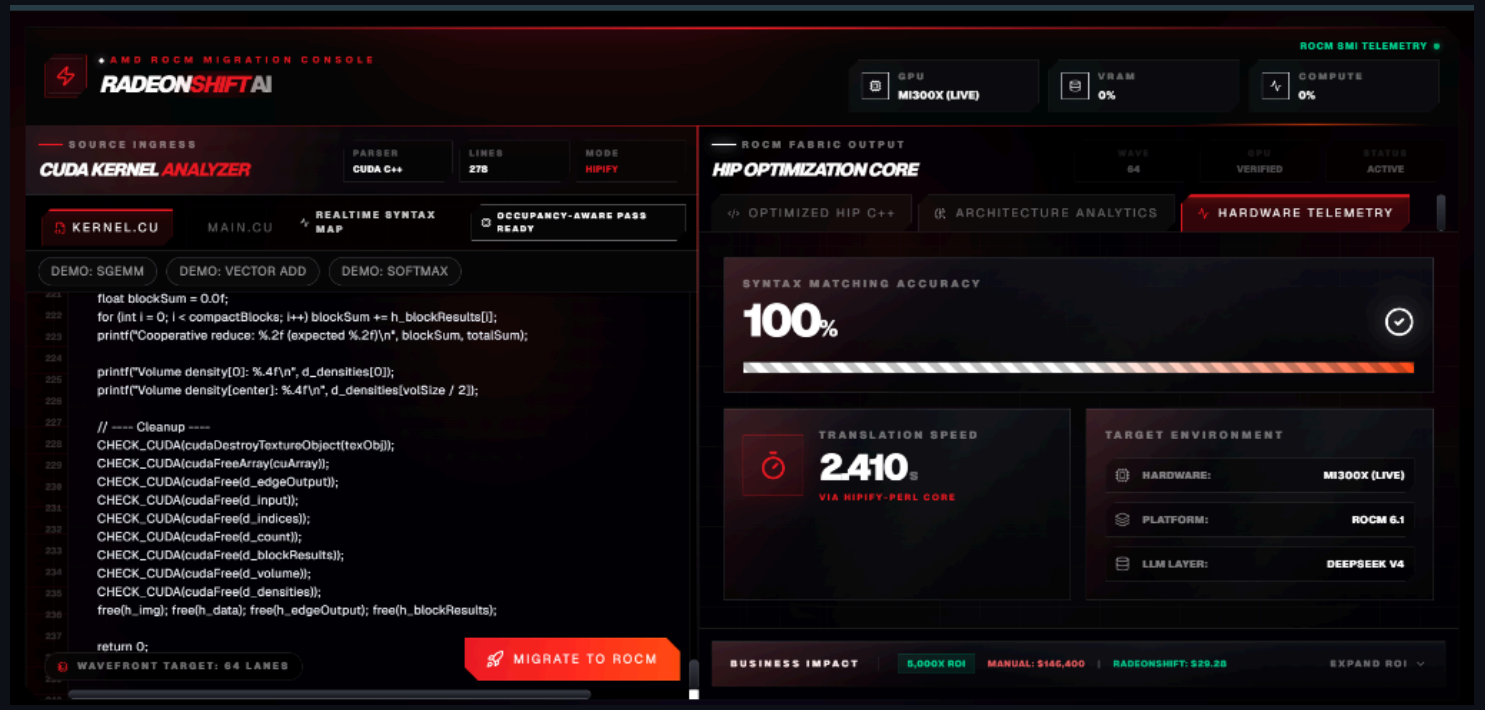

The MoA Audit Scorecard evaluates the code. Here, the readiness score is 100/100, proving high portability. Our dual AI agents verify there are no hidden PTX risks or lock-ins.

# Step 4: Live Hardware Telemetry




Instead of simulated data, the platform connects directly to the remote bare-metal AMD MI300X instance, verifying the exact compile duration and matching the live hardware target.

# Step 5: Bare-Metal Benchmark Execution




Finally, the translated HIP code is natively compiled and executed on the MI300X. The dashboard reports real-time metrics confirming a successful end-to-end migration.

# Business Value & ROI

## The 5,000x ROI Model

- **Manual Migration:** 244 Kernels × 4 hrs = 976 Engineer-Hours (~$146,000, based on $150/hr senior GPU engineer rate)
- **RadeonShift Migration:** 244 Kernels × 0.12 compute cost = ~$29
- **Time Savings:** Months reduced to minutes.
- **TAM:** $25B+ (Global AI Hardware & Software Services Market)

## The AMD Infrastructure Story

- **Hardware Target:** Optimized for AMD Instinct MI300X
- **Software Stack:** Built on ROCm 6.x APIs and `hipcc`
- **AI Acceleration:** MoA pipeline runs on Fireworks AI, which provides AMD Instinct GPU-powered inference
- **Honest Fallbacks:** If the backend lacks ROCm hardware, the platform reports "Hardware Unavailable" without fabricating metrics

# Engineering Retrospective & Challenges

### Bridging a Serverless Frontend with Bare-Metal Hardware

1. **Live Telemetry:** Defeated Next.js caching via `force-dynamic` to stream real-time MI300X metrics.
2. **Defensive APIs:** Built robust frontend parsing to gracefully handle sudden JSON schema changes.
3. **CORS Routing:** Bypassed strict mixed-content blocks by tunneling through Pinggy and Next.js `rewrites()`.
4. **Transparent Debugging:** Overhauled error handling to surface remote Python stack traces in the UI.
5. **Deterministic AI Prompts:** Forced LLMs to generate explicit confirmations for clean code, preventing UI state collapses.

# Closing & Team

**RadeonShift AI** is bridging the gap between legacy NVIDIA codebases and the future of AMD compute.

- **GitHub Repository:** shashankh3/RadeonShift-AI
- **Live Demo:** radeon-shift-ai.vercel.app

# Thank You!

**Shashank Hirwani**
Unknown Hacker (shashankh366207)